Package 'breedtools'

June 24, 2026

Title Tools for Multi-Environment Breedbase Phenotype Analysis

Version 0.0.0.9000

Depends R (>= 4.1.0)

Author Christiano Simoes [aut, cre]

Maintainer Christiano Simoes <ccsimoes263@gmail.com>

Description Loads Breedbase-style phenotype data from Excel or CSV files, prepares it for multi-environment analysis, checks environment variance compatibility with Hartley's Fmax test, and groups environments with Lin-style genotype-by-environment interaction tests.

License GPL-3

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 8.0.0

Imports dplyr,
tidyr,
purrr,
readxl,
leaflet,
htmltools,
tibble,
tools,
graphics,
grDevices,
jsonlite,
stats,
utils

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

Contents

add_environment_metadata 3

add_pairwise_environment_metadata . . . 3
analyze_augmented_row_column_design . . . 4
analyze_environment_anova . . . 5
analyze_one_environment . . . 6
augmented_prep_concurrence . . . 6
augmented_prep_design . . . 7
augmented_row_column_capacity . . . 9
augmented_row_column_design . . . 10
calculate_environment_stratification_anova . . . 11
complete_environment_info . . . 12
detect_design . . . 12
empty_anova . . . 13
empty_group_membership . . . 13
empty_group_summary . . . 13
empty_pairwise . . . 14
empty_ungrouped . . . 14
environment_summary_by_group . . . 14
find_column . . . 15
get_compatible_groups . . . 15
get_maximal_groups . . . 16
guess_column . . . 16
harmonic_mean . . . 17
hartley_critical_sim . . . 17
hartley_fmax_test . . . 18
lin_error_mse . . . 18
lin_group_environments . . . 19
lin_ss_ge . . . 19
lin_test_group . . . 20
make_environment_groups . . . 20
map_environment_groups . . . 21
normalize_column_name . . . 22
plot_augmented_prep_design . . . 22
plot_augmented_row_column_design . . . 24
prepare_breedbase_pheno . . . 26
prepare_design_factors . . . 27
prepare_environment_stratification_data . . . 27
print.augmented_row_column_analysis . . . 28
read_breedbase_excel . . . 28
read_breedbase_file . . . 29
read_locations . . . 30
run_breedbase_environment_stratification . . . 31
run_breedbase_variance_check . . . 32
run_environment_stratification . . . 33

Index **35**

add_environment_metadata

Add environment metadata to result table

Description

Add environment metadata to result table

Usage

```
add_environment_metadata(results, env_info, env_col = "environment")
```

Arguments

`results`	Result data frame.
`env_info`	Environment metadata table.
`env_col`	Name of the environment column in `results`.

Value

Joined data frame.

add_pairwise_environment_metadata

Add environment metadata to pairwise Lin results

Description

Add environment metadata to pairwise Lin results

Usage

```
add_pairwise_environment_metadata(pairwise, env_info)
```

Arguments

`pairwise`	Pairwise result table.
`env_info`	Environment metadata table.

Value

Pairwise table with metadata.

analyze_augmented_row_column_design
Analyze an augmented row-column design

Description

Fits the fixed-effects model: y = mean + rowgroup + colgroup + rowgroup:colgroup + rowgroup:row + colgroup:col + genotype + error. The formatted ANOVA table reports degrees of freedom from augmented row-column design formulas using field rows (k), field columns (s), row groups (gk), column groups (gs), check cultivars (vc), and unreplicated entries (ve).

Usage

```
analyze_augmented_row_column_design(
  data,
  response,
  genotype_col = "all_entries",
  rowgroup_col = "rowgroup",
  colgroup_col = "colgroup",
  row_col = "row",
  col_col = "col",
  type_col = "type",
  control_col = "is_control"
)
```

Arguments

data	A complete augmented row-column design data frame, usually design$design from augmented_row_column_design(), with a response column added.
response	Name of the numeric response column.
genotype_col	Name of the genotype/treatment column.
rowgroup_col	Name of the row-group column.
colgroup_col	Name of the column-group column.
row_col	Name of the field row column.
col_col	Name of the field column column.
type_col	Optional entry type column. Used only for genotype summaries.
control_col	Optional control flag column. Used only for genotype summaries.

Value

A list with the analysis input, fitted model, model formula, ANOVA table, formatted ANOVA table, plot-level fitted values and residuals, genotype summaries, and model diagnostics.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(108)),
  controls = paste0("Check", seq_len(4)),
  rows_in_field = 12,
  cols_in_field = 12,
  rows_per_block = 4,
  cols_per_block = 4,
  n_candidates = 3,
  seed = 123
)

design_data <- design$design
design_data$yield <- rnorm(nrow(design_data), mean = 100, sd = 10)
analysis <- analyze_augmented_row_column_design(design_data, response = "yield")
analysis$anova
```

`analyze_environment_anova`

Run individual ANOVA by environment

Description

Run individual ANOVA by environment

Usage

```
analyze_environment_anova(pheno)
```

Arguments

pheno	Standardized phenotype data from `prepare_breedbase_pheno()`.

Value

A tibble with one ANOVA summary row per environment.

Examples

```
df <- data.frame(
  environment = factor(rep(c("Loc1", "Loc2"), each = 8)),
  genotype = factor(rep(rep(c("G1", "G2"), each = 4), times = 2)),
  replicate = factor(rep(rep(c(1, 2), each = 2), times = 4)),
  block = factor(1),
  value = c(10, 11, 12, 13, 14, 15, 16, 17,
            15, 16, 17, 18, 19, 20, 21, 22)
)
```

```
analyze_environment_anova(df)
```

`analyze_one_environment` *Analyze one environment using ANOVA*

Description

Runs an individual ANOVA for one environment. If more than one replicate is present, the model is `value ~ replicate + genotype`; otherwise, the model is `value ~ genotype`.

Usage

```
analyze_one_environment(df)
```

Arguments

`df`	A data frame containing one environment with columns `environment`, `genotype`, `replicate`, and `value`.

Value

A tibble with ANOVA summary statistics.

`augmented_prep_concurrence` *Calculate pairwise concurrence for a p-rep design*

Description

Counts how many times each pair of entries appears together in the same incomplete block across all locations.

Usage

```
augmented_prep_concurrence(design, treatments = NULL)
```

Arguments

`design`	A p-rep design data frame from `augmented_prep_design()`.
`treatments`	Optional treatment names defining matrix order.

Value

A symmetric integer matrix of pairwise block concurrences.

augmented_prep_design *Generate an augmented p-rep design*

Description

Creates an augmented partially replicated (p-rep) design inspired by the Williams, Piepho, and Whitaker approach. Instead of replicated checks, a subset of test entries is replicated at each location, while all entries appear once at each location and the additional replicated entries are placed into incomplete blocks. Candidate designs are scored to reduce pairwise block concurrence, especially avoiding pairs that occur together more than once.

Usage

```
augmented_prep_design(
  treatments,
  locations,
  block_size = 8,
  prep_prop = NULL,
  n_duplicate_per_location = NULL,
  replications_per_repeated_entry = 2,
  n_blocks_per_location = NULL,
  n_candidates = 1000,
  seed = NULL,
  field_cols = NULL,
  serpentine = TRUE,
  output_file = NULL
)
```

Arguments

`treatments`	Character vector of entry names.
`locations`	Character vector of location names.
`block_size`	Maximum final incomplete block size. Can be a single integer or a named vector indexed by location. Actual block sizes may differ by one plot when totals do not divide evenly.
`prep_prop`	Optional proportion of entries to duplicate per location. If `NULL`, entries are split across locations so each entry is duplicated at approximately one location.
`n_duplicate_per_location`	Optional number of entries duplicated at each location. Can be a single integer, a vector matching `locations`, or a named vector indexed by location.
`replications_per_repeated_entry`	Total number of plots for entries selected for p-rep replication at a location. The default `2` gives one base plot plus one extra plot. Can be a single integer or a named vector indexed by location.

`n_blocks_per_location`
: Optional number of incomplete blocks at each location. If `NULL`, the number of blocks is the minimum needed so no block is larger than `block_size`. Can be a single integer or a named vector indexed by location.

`n_candidates`
: Number of candidate designs to generate and score.

`seed`
: Optional random seed.

`field_cols`
: Optional number of field columns for row/column layout. Can be a single integer or a named vector indexed by location.

`serpentine`
: Logical. If `TRUE`, row/column coordinates use serpentine ordering within each location.

`output_file`
: Optional tab-delimited output file for the complete design.

Details

The incomplete block structure is independent from the field geometry: `block_size` and `n_blocks_per_location` define block membership, while `field_cols` only lays the ordered plots into rows and columns. Use a `field_cols` value different from the final block sizes when more than one block should appear in a field row or a block should span rows.

Value

A list with design inputs, summary diagnostics, complete plot-level design, Breedbase-compatible design columns, block summary, replication summary, repeated-entry groups, and pairwise concurrence matrix. In the complete design, `repeated_at` is the location where the entry receives its extra p-rep plot(s), or `"not_repeated"` when the entry is not selected for extra replication.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = c(Loc1 = 5, Loc2 = 5, Loc3 = 6),
  n_blocks_per_location = c(Loc1 = 4, Loc2 = 4, Loc3 = 4),
  n_duplicate_per_location = c(Loc1 = 4, Loc2 = 2, Loc3 = 4),
  replications_per_repeated_entry = c(Loc1 = 2, Loc2 = 3, Loc3 = 2),
  n_candidates = 5,
  seed = 123,
  field_cols = 8
)

design$summary
head(design$design)
```

augmented_row_column_capacity *Calculate augmented row-column design capacity*

Description

Calculate augmented row-column design capacity

Usage

```
augmented_row_column_capacity(
  rows_in_field,
  cols_in_field,
  rows_per_block,
  cols_per_block,
  n_controls
)
```

Arguments

rows_in_field	Number of rows in the whole field.
cols_in_field	Number of columns in the whole field.
rows_per_block	Number of rows in each row-column block.
cols_per_block	Number of columns in each row-column block.
n_controls	Number of replicated checks/controls. Each block receives one copy of each control.

Value

A one-row tibble with number of blocks, total plots, check plots, and entry plots available for unreplicated treatments.

Examples

```
augmented_row_column_capacity(
  rows_in_field = 6,
  cols_in_field = 6,
  rows_per_block = 3,
  cols_per_block = 3,
  n_controls = 2
)
```

augmented_row_column_design *Generate an augmented row-column field design*

Description

Randomizes unreplicated treatments and replicated controls in an augmented row-column design. Each row-column block receives one copy of each control, and controls are constrained so they do not share a row or column within a block. Candidate layouts are scored and the best candidate is returned.

Usage

```
augmented_row_column_design(
  treatments,
  controls,
  rows_in_field,
  cols_in_field,
  rows_per_block,
  cols_per_block,
  plot_type = c("serpentine", "cartesian"),
  n_candidates = 1000,
  seed = NULL,
  output_file = NULL
)
```

Arguments

treatments	Character vector of unreplicated treatment/entry names.
controls	Character vector of replicated check/control names.
rows_in_field	Number of rows in the whole field.
cols_in_field	Number of columns in the whole field.
rows_per_block	Number of rows in each row-column block.
cols_per_block	Number of columns in each row-column block.
plot_type	Plot numbering order. Use "serpentine" or "cartesian".
n_candidates	Number of candidate layouts to generate and score.
seed	Optional random seed.
output_file	Optional path for writing the complete design as a tab-delimited file.

Value

A list with:

input Validated design inputs.

summary Design size and optimization summary.

design Complete plot-level design with row, column, block, entry, control flag, and plot order.

breedbase_design Breedbase-compatible columns: `plots`, `block`, `all_entries`, `rep`, and `is_control`.

Examples

```
treatments <- paste0("G", seq_len(28))
controls <- c("Check1", "Check2")

design <- augmented_row_column_design(
  treatments = treatments,
  controls = controls,
  rows_in_field = 6,
  cols_in_field = 6,
  rows_per_block = 3,
  cols_per_block = 3,
  n_candidates = 5,
  seed = 123
)

head(design$design)
design$summary
```

`calculate_environment_stratification_anova`
Calculate design-aware ANOVA

Description

Calculate design-aware ANOVA

Usage

```
calculate_environment_stratification_anova(data)
```

Arguments

`data`	Standardized phenotype data.

Value

ANOVA table as a data frame.

complete_environment_info *Complete environment metadata*

Description

Complete environment metadata

Usage

```
complete_environment_info(env_info)
```

Arguments

env_info	Data frame with environment, location, trial, and year.

Value

Data frame with an added environment_label column.

detect_design *Detect field trial design for environment stratification*

Description

Detects whether the data look like CRD, RCBD, row-column, alpha-lattice, or augmented/block-only design.

Usage

```
detect_design(data)
```

Arguments

data	Standardized phenotype data.

Value

A list with design, formula, and message.

empty_anova *Empty ANOVA result table*

Description

Empty ANOVA result table

Usage

```
empty_anova()
```

Value

Empty data frame.

empty_group_membership *Empty group membership table*

Description

Empty group membership table

Usage

```
empty_group_membership()
```

Value

Empty data frame.

empty_group_summary *Empty group summary table*

Description

Empty group summary table

Usage

```
empty_group_summary()
```

Value

Empty data frame.

empty_pairwise *Empty pairwise Lin result table*

Description

Empty pairwise Lin result table

Usage

```
empty_pairwise()
```

Value

Empty data frame.

empty_ungrouped *Empty ungrouped environment table*

Description

Empty ungrouped environment table

Usage

```
empty_ungrouped()
```

Value

Empty data frame.

environment_summary_by_group *Summarize environment names by group*

Description

Summarize environment names by group

Usage

```
environment_summary_by_group(group_summary, group_membership)
```

Arguments

group_summary	Group summary table.
group_membership	Group membership table.

Value

Group summary with display metadata.

---

find_column *Find the first matching column*

Description

Find the first matching column

Usage

```
find_column(data, candidates, label = "column")
```

Arguments

data	A data frame.
candidates	Candidate column names.
label	Label used in the error message.

Value

The first matching column name.

---

get_compatible_groups *Get compatible environment groups*

Description

Get compatible environment groups

Usage

```
get_compatible_groups(group_tests)
```

Arguments

group_tests	Output from make_environment_groups().

Value

A tibble containing only homogeneous groups.

get_maximal_groups *Select maximal compatible environment groups*

Description

Removes smaller compatible groups that are fully contained inside larger compatible groups.

Usage

```
get_maximal_groups(compatible_groups)
```

Arguments

compatible_groups	Output from get_compatible_groups().

Value

A tibble with maximal compatible groups only.

guess_column *Guess a column name from possible Breedbase column names*

Description

Guess a column name from possible Breedbase column names

Usage

```
guess_column(df, possible_names)
```

Arguments

df	A data frame.
possible_names	A character vector of possible column names.

Value

A column name if found, otherwise NULL.

Examples

```
df <- data.frame(germplasmName = "G1", phenotype = 10)
guess_column(df, c("germplasmName", "accession_name"))
```

harmonic_mean *Calculate harmonic mean*

Description

Calculate harmonic mean

Usage

```
harmonic_mean(x)
```

Arguments

x	Numeric vector.

Value

Harmonic mean.

hartley_critical_sim *Simulate Hartley Fmax critical value*

Description

Estimates the Hartley Fmax critical value using simulation. This version can handle different residual degrees of freedom across environments.

Usage

```
hartley_critical_sim(df_error_vec, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

df_error_vec	Numeric vector of residual degrees of freedom.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations. Default is 100000.
seed	Random seed.

Value

A numeric critical value.

hartley_fmax_test *Hartley Fmax test for residual variance homogeneity*

Description

Hartley Fmax test for residual variance homogeneity

Usage

```
hartley_fmax_test(env_anova, alpha = 0.05, n_sim = 1e+05, seed = 123)
```

Arguments

env_anova	A data frame containing environment-level ANOVA results. It must include columns environment, ms_error, and df_error.
alpha	Significance level. Default is 0.05.
n_sim	Number of simulations for the critical value.
seed	Random seed.

Value

A tibble with observed Fmax, critical Fmax, and homogeneity decision.

lin_error_mse *Calculate residual error MSE for a Lin environment group*

Description

Calculate residual error MSE for a Lin environment group

Usage

```
lin_error_mse(data, envs, common_genotypes)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments.
common_genotypes	Character vector of common genotypes.

Value

A list with MSE, residual df, and message.

lin_group_environments *Group environments using Lin-style GE compatibility testing*

Description

Group environments using Lin-style GE compatibility testing

Usage

```
lin_group_environments(data, alpha = 0.05)
```

Arguments

data	Standardized phenotype data.
alpha	Significance level.

Value

A list with pairwise tests, group summary, group membership, and ungrouped environments.

lin_ss_ge *Calculate Lin genotype-by-environment sum of squares*

Description

Calculate Lin genotype-by-environment sum of squares

Usage

```
lin_ss_ge(data, envs)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments to test.

Value

A list with a summary table and common genotype names.

lin_test_group *Test one Lin environment group*

Description

Test one Lin environment group

Usage

```
lin_test_group(data, envs, alpha = 0.05)
```

Arguments

data	Standardized phenotype data.
envs	Character vector of environments.
alpha	Significance level.

Value

Data frame with Lin GE test result.

make_environment_groups *Test all compatible environment groups using Hartley Fmax*

Description

Tests all combinations of environments and identifies groups with homogeneous residual variances.

Usage

```
make_environment_groups(
  env_anova,
  alpha = 0.05,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123
)
```

Arguments

env_anova	A data frame from analyze_environment_anova().
alpha	Significance level.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed.

Value

A tibble with all tested groups.

`map_environment_groups` *Create a map of environment groups*

Description

Creates an interactive `leaflet` map where locations belonging to the same environment group use the same marker color. Ungrouped environments are shown with `ungrouped_color`.

Usage

```
map_environment_groups(
  stratification_result,
  locations,
  ignore_case = FALSE,
  group_palette = NULL,
  ungrouped_color = "#7A7A7A",
  match_by = "location",
  strict = TRUE
)
```

Arguments

`stratification_result`	Result from `run_environment_stratification()` or `run_breedbase_environment_stratificatior`
`locations`	Location coordinates from `read_locations()`, a location file path, or a data frame with location, latitude, and longitude columns.
`ignore_case`	Logical. If `TRUE`, location matching ignores case.
`group_palette`	Optional vector of colors used for environment groups.
`ungrouped_color`	Color for ungrouped environments.
`match_by`	Name from the stratification result used to match the `location` column in `locations`. Use `"location"` when coordinates are named by location, `"environment"` when coordinates are named by the constructed environment identifier, or `"environment_label"` when coordinates are named by display labels.
`strict`	Logical. If `TRUE`, stop when any environment location has no matching coordinates.

Value

A `leaflet` htmlwidget.

Examples

```
## Not run:
result <- run_environment_stratification("phenotype.csv", trait = "yield")
locations <- read_locations("locations.csv", ignore_case = TRUE)
map_environment_groups(result, locations, ignore_case = TRUE)

## End(Not run)
```

`normalize_column_name` *Normalize Breedbase-style column names*

Description

Removes ontology suffixes such as `.CO...` or `|CO_...` and replaces dots with spaces.

Usage

```
normalize_column_name(x)
```

Arguments

`x`	Character vector of column names.

Value

Normalized character vector.

`plot_augmented_prep_design`
Plot an augmented p-rep design grid

Description

Draws one field grid per location for an augmented p-rep design, with plot cells, row and column labels, and thick outlines around incomplete blocks.

Usage

```
plot_augmented_prep_design(
  design,
  locations = NULL,
  label = c("all_entries", "entry_type", "repeated_at", "block", "none"),
  fill = c("entry_type", "is_extra_repeat", "repeated_at", "block"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
  main = NULL,
  legend = TRUE,
  ...
)
```

Arguments

design	An object returned by augmented_prep_design() or its design data frame.
locations	Optional character vector of locations to plot. Defaults to all locations in design order.
label	Cell label to show. Use "all_entries", "entry_type", "repeated_at", "block", or "none".
fill	Cell coloring variable. Use "entry_type", "is_extra_repeat", "repeated_at", or "block".
show_plot_labels	Logical. If TRUE, labels are printed inside cells unless label = "none".
show_block_labels	Logical. If TRUE, block numbers are printed. Defaults to FALSE so block labels do not overlap accession labels.
block_label_position	Position for block labels when show_block_labels = TRUE. Use "background" for a faint watermark behind accession labels or "center" for a regular center label.
label_cex	Optional text size for plot-cell labels.
block_label_cex	Optional text size for block labels.
block_label_col	Color for block labels.
block_lwd	Line width for block boundaries.
main	Optional plot title. If multiple locations are plotted, a single value is combined with the location name; a vector matching the number of locations is used panel by panel.
legend	Logical. If TRUE, draw a legend for cell colors.
...	Additional arguments passed to graphics::plot.window().

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_prep_design(
  treatments = paste0("G", seq_len(12)),
  locations = c("Loc1", "Loc2", "Loc3"),
  block_size = 5,
  n_candidates = 5,
  seed = 123,
  field_cols = 5
)

plot_augmented_prep_design(design)
```

---

plot_augmented_row_column_design

Plot an augmented row-column design grid

Description

Draws a field grid for an augmented row-column design, with plot cells, row and column labels, and thick outlines around row-column blocks.

Usage

```
plot_augmented_row_column_design(
  design,
  label = c("all_entries", "type", "block", "none"),
  fill = c("type", "block", "rowgroup", "colgroup", "is_control"),
  show_plot_labels = TRUE,
  show_block_labels = FALSE,
  block_label_position = c("background", "center"),
  label_cex = NULL,
  block_label_cex = NULL,
  block_label_col = NULL,
  block_lwd = 2,
  main = NULL,
  legend = TRUE,
  ...
)
```

Arguments

design	An object returned by augmented_row_column_design() or a design data frame with row, column, and block columns.
label	Cell label to show. Use "all_entries", "type", "block", or "none".
fill	Cell coloring variable. Use "type", "block", "rowgroup", "colgroup", or "is_control".
show_plot_labels	Logical. If TRUE, labels are printed inside cells unless label = "none".
show_block_labels	Logical. If TRUE, block numbers are printed. Defaults to FALSE so block labels do not overlap accession labels.
block_label_position	Position for block labels when show_block_labels = TRUE. Use "background" for a faint watermark behind accession labels or "center" for a regular center label.
label_cex	Optional text size for plot-cell labels.
block_label_cex	Optional text size for block labels.
block_label_col	Color for block labels.
block_lwd	Line width for block boundaries.
main	Optional plot title.
legend	Logical. If TRUE, draw a legend for cell colors.
...	Additional arguments passed to graphics::plot.window().

Value

Invisibly returns the plotted design data frame with internal plotting coordinates.

Examples

```
design <- augmented_row_column_design(
  treatments = paste0("G", seq_len(28)),
  controls = c("Check1", "Check2"),
  rows_in_field = 6,
  cols_in_field = 6,
  rows_per_block = 3,
  cols_per_block = 3,
  n_candidates = 3,
  seed = 123
)

plot_augmented_row_column_design(design, label = "all_entries")
```

prepare_breedbase_pheno *Prepare Breedbase phenotype data for multi-environment analysis*

Description

This function standardizes Breedbase phenotype data into five columns: `environment`, `genotype`, `replicate`, `block`, and `value`.

Usage

```
prepare_breedbase_pheno(
  pheno_raw,
  trait_col,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL
)
```

Arguments

`pheno_raw`	A raw Breedbase phenotype data frame, usually read from a Breedbase Excel file.
`trait_col`	Name of the trait column to analyze.
`location_col`	Optional location/environment column name. If `NULL`, the function tries to guess it.
`genotype_col`	Optional genotype/accession column name. If `NULL`, the function tries to guess it.
`rep_col`	Optional replicate column name.
`block_col`	Optional block column name.

Value

A tibble with standardized phenotype data.

Examples

```
df <- data.frame(
  location = c("Loc1", "Loc1", "Loc2", "Loc2"),
  germplasmName = c("G1", "G2", "G1", "G2"),
  rep = c(1, 1, 1, 1),
  phenotype = c(10, 12, 15, 14)
)

prepare_breedbase_pheno(df, trait_col = "phenotype")
```

prepare_design_factors *Prepare design factors*

Description

Prepare design factors

Usage

```
prepare_design_factors(data)
```

Arguments

data	Standardized phenotype data.

Value

Data frame with design columns converted to factors.

prepare_environment_stratification_data *Prepare phenotype data for Lin environment stratification*

Description

Reads a Breedbase-style phenotype data frame and standardizes it for environment stratification.

Usage

```
prepare_environment_stratification_data(pheno, trait, normalize_names = TRUE)
```

Arguments

pheno	A phenotype data frame.
trait	Trait column name.
normalize_names	Logical. If TRUE, normalizes column names.

Value

A list with standardized phenotype data, environment metadata, and selected trait column.

print.augmented_row_column_analysis *Print augmented row-column analysis*

Description

Print augmented row-column analysis

Usage

```
## S3 method for class 'augmented_row_column_analysis'
print(x, ...)
```

Arguments

x	An object returned by analyze_augmented_row_column_design().
...	Additional arguments passed to print().

Value

Invisibly returns x.

read_breedbase_excel *Read a Breedbase Excel phenotype file*

Description

read_breedbase_excel() is deprecated. Use read_breedbase_file() for .xlsx, .xls, and .csv files.

Usage

```
read_breedbase_excel(file, ...)
```

Arguments

file	Path to the Excel file.
...	Additional arguments passed to read_breedbase_file().

Value

A tibble containing combined data from all non-empty sheets.

Examples

```
## Not run:
pheno_raw <- read_breedbase_file("breedbase_trials_pheno.xlsx")

## End(Not run)
```

`read_breedbase_file` *Read phenotype or design data from Excel, CSV, or tab-delimited files*

Description

Loads phenotype or field-design data from `.xlsx`, `.xls`, `.csv`, `.tsv`, `.txt`, or `.design` files. Excel files are read from all non-empty sheets and combined into one tibble with a `.sheet` column identifying the source sheet. CSV files are comma-delimited; `.tsv`, `.txt`, and `.design` files are tab-delimited.

Usage

```
read_breedbase_file(file, sheet = NULL, ...)
```

Arguments

`file`	Path to the phenotype file.
`sheet`	Optional Excel sheet name or number. If `NULL`, all non-empty Excel sheets are combined.
`...`	Additional arguments passed to `readxl::read_excel()` for Excel files, `utils::read.csv()` for CSV files, or `utils::read.table()` for tab-delimited files.

Value

A tibble containing the loaded phenotype data.

Examples

```
## Not run:
pheno <- read_breedbase_file("breedbase_trials_pheno.xlsx")
pheno_csv <- read_breedbase_file("breedbase_trials_pheno.csv")
design <- read_breedbase_file("augmented_row_column.design")

## End(Not run)
```

`read_locations` *Load location coordinates*

Description

Loads location metadata from a `.xlsx`, `.xls`, or `.csv` file, or from an already loaded data frame. The returned table has standardized columns: `location`, `latitude`, and `longitude`.

Usage

```
read_locations(
  file,
  location_col = NULL,
  latitude_col = NULL,
  longitude_col = NULL,
  phenotype_data = NULL,
  phenotype_location_col = NULL,
  match_by = "location",
  reference_locations = NULL,
  ignore_case = FALSE,
  sheet = NULL,
  ...
)
```

Arguments

`file`	Path to a location file, or an already loaded data frame.
`location_col`	Optional location name column.
`latitude_col`	Optional latitude column.
`longitude_col`	Optional longitude column.
`phenotype_data`	Optional raw phenotype data frame, prepared phenotype data frame, or result from `run_environment_stratification()`. If supplied, location names are extracted and checked against the location file.
`phenotype_location_col`	Optional location column in `phenotype_data`.
`match_by`	Column to extract when `phenotype_data` is a stratification result. Use `"location"`, `"environment"`, or `"environment_label"`.
`reference_locations`	Optional character vector of expected location names. If supplied, every reference location must be present in `file`.
`ignore_case`	Logical. If `TRUE`, location matching ignores case.
`sheet`	Optional Excel sheet name or number passed to `read_breedbase_file()`.
`...`	Additional arguments passed to `read_breedbase_file()`.

Value

A tibble with `location`, `latitude`, and `longitude`.

Examples

```
locations <- data.frame(
  location = c("Loc1", "Loc2"),
  latitude = c(35.1, 36.2),
  longitude = c(-78.9, -79.4)
)

read_locations(locations)
```

`run_breedbase_environment_stratification` *Run Lin-style environment stratification from a Breedbase phenotype file*

Description

Reads a Breedbase phenotype file, prepares the phenotype data, detects the experimental design, calculates ANOVA, and groups environments using a Lin-style genotype-by-environment interaction test.

Usage

```
run_breedbase_environment_stratification(
  file,
  trait_col,
  alpha = 0.05,
  normalize_names = TRUE
)
```

Arguments

`file`	Path to a `.xlsx`, `.xls`, or `.csv` phenotype file, or an already loaded data frame.
`trait_col`	Trait column name.
`alpha`	Significance level. Default is `0.05`.
`normalize_names`	Logical. If `TRUE`, normalize Breedbase-style column names.

Value

A list containing standardized phenotype data, environment metadata, ANOVA results, pairwise environment tests, group summary, group membership, ungrouped environments, summary, and message.

Examples

```
## Not run:
result <- run_breedbase_environment_stratification(
  file = "breedbase_trials_pheno.xlsx",
  trait_col = "phenotype",
  alpha = 0.05
)

result$group_summary
result$group_membership

## End(Not run)
```

`run_breedbase_variance_check` *Run Breedbase multi-location phenotype variance check*

Description

Reads a Breedbase phenotype file, runs individual ANOVA by environment, performs Hartley Fmax tests, and identifies compatible environment groups for joint analysis.

Usage

```
run_breedbase_variance_check(
  file,
  trait_col,
  alpha = 0.05,
  location_col = NULL,
  genotype_col = NULL,
  rep_col = NULL,
  block_col = NULL,
  max_group_size = Inf,
  n_sim = 1e+05,
  seed = 123,
  export = FALSE,
  output_dir = "."
)
```

Arguments

`file`	Path to a `.xlsx`, `.xls`, or `.csv` phenotype file, or an already loaded data frame.
`trait_col`	Trait column to analyze.
`alpha`	Significance level for Hartley Fmax test.
`location_col`	Optional location/environment column.

genotype_col	Optional genotype/accession column.
rep_col	Optional replicate column.
block_col	Optional block column.
max_group_size	Maximum environment group size to test.
n_sim	Number of simulations for Hartley critical value.
seed	Random seed.
export	Logical. If TRUE, writes CSV files.
output_dir	Directory where CSV files should be written.

Value

A list containing cleaned data, ANOVA results, Hartley test, all group tests, compatible groups, maximal compatible groups, and detected columns.

Examples

```
## Not run:
result <- run_breedbase_variance_check(
  file = "breedbase_trials_pheno.xlsx",
  trait_col = "phenotype"
)

result$env_anova
result$maximal_compatible_groups

## End(Not run)
```

---

run_environment_stratification

Run Lin-style environment stratification

Description

This function reads phenotype data, detects design structure, calculates ANOVA, tests genotype-by-environment interaction for environment groups, and returns compatible environment groups.

Usage

```
run_environment_stratification(
  phenotype_file,
  trait,
  alpha = 0.05,
  sep = NULL,
  normalize_names = TRUE,
  export_json = FALSE,
```

```
  output_dir = ".",
  prefix = "environment_stratification"
)
```

Arguments

phenotype_file	Path to a .xlsx, .xls, or .csv phenotype file, or an already loaded data frame.
trait	Trait column name.
alpha	Significance level. Default is 0.05.
sep	Deprecated. File delimiters are detected from the extension by read_breedbase_file().
normalize_names	Logical. If TRUE, normalize Breedbase column names.
export_json	Logical. If TRUE, writes JSON result files.
output_dir	Output directory for JSON files.
prefix	Prefix for JSON output files.

Value

A list with summary, ANOVA results, pairwise tests, group summary, group membership, ungrouped environments, and message.

Examples

```
## Not run:
result <- run_environment_stratification(
  phenotype_file = "phenotype.tsv",
  trait = "plant height",
  alpha = 0.05
)

result$group_summary
result$group_membership

## End(Not run)
```

Index

add_environment_metadata, 3
add_pairwise_environment_metadata, 3
analyze_augmented_row_column_design, 4
analyze_environment_anova, 5
analyze_environment_anova(), *20*
analyze_one_environment, 6
augmented_prep_concurrence, 6
augmented_prep_design, 7
augmented_row_column_capacity, 9
augmented_row_column_design, 10

calculate_environment_stratification_anova, 11
complete_environment_info, 12

detect_design, 12

empty_anova, 13
empty_group_membership, 13
empty_group_summary, 13
empty_pairwise, 14
empty_ungrouped, 14
environment_summary_by_group, 14

find_column, 15

get_compatible_groups, 15
get_compatible_groups(), *16*
get_maximal_groups, 16
guess_column, 16

harmonic_mean, 17
hartley_critical_sim, 17
hartley_fmax_test, 18

lin_error_mse, 18
lin_group_environments, 19
lin_ss_ge, 19
lin_test_group, 20

make_environment_groups, 20
make_environment_groups(), *15*
map_environment_groups, 21

normalize_column_name, 22

plot_augmented_prep_design, 22
plot_augmented_row_column_design, 24
prepare_breedbase_pheno, 26
prepare_breedbase_pheno(), *5*
prepare_design_factors, 27
prepare_environment_stratification_data, 27
print.augmented_row_column_analysis, 28

read_breedbase_excel, 28
read_breedbase_file, 29
read_locations, 30
run_breedbase_environment_stratification, 31
run_breedbase_variance_check, 32
run_environment_stratification, 33